CO 1 – LAB EXERCISES

ITA04 – Statistics with R Programming

Submitted by:
R. Sharmila

Register Number: 192521213

LAB EXERCISE 1: R and RStudio Environment

Question:

Explore the R and RStudio environment before writing a program. Identify the console, script editor, environment/workspace pane, files pane and help system. Run a simple command, inspect the current R version and working directory, and open the help page for mean(). Record what each environment feature is used for.

Aim:

To explore, identify, and understand the components of the R and RStudio environment, and run fundamental environment commands.

Algorithm:

1. Launch RStudio and observe the primary user interface panes.
2. Identify the Script Editor (top-left) for writing and saving multi-line R scripts.
3. Identify the R Console (bottom-left) for direct execution of interactive commands.
4. Identify the Environment/Workspace pane (top-right) which tracks active variables and data objects.
5. Identify the Files, Plots, Packages, and Help pane (bottom-right) for file directory navigation and documentation viewing.
6. Execute commands in the console to fetch the active R software version, print the present working directory paths, open the official help specification page for the built-in mean() function, and perform a basic printing command.

Input:

```
# 1. Display current R version details
print(R.version.string)

# 2. Check current working directory
current_dir <- getwd()
print(paste("Current Working Directory:", current_dir))
```

```
# 3. Open documentation help page for mean()
help(mean)

# 4. Run a simple baseline command
message <- "R and RStudio Environment Initialized Successfully!"
print(message)
```

Output Screenshot

```
> print(R.version.string)
[1] "R version 4.6.1 (2026-06-24 ucrt)"
>
> current_dir <- getwd()
> print(paste("Current Working Directory:", current_dir))
[1] "Current Working Directory: C:/Users/arcot/OneDrive - vit.ac.in/Documents"
>
> help(mean)
>
> message <- "R and RStudio Environment Initialized Successfully!"
> print(message)
[1] "R and RStudio Environment Initialized Successfully!"
```

Result:

The program for R and RStudio Environment was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 2: Arithmetic Operations

Question:

Write an R program that stores two numbers in variables and performs addition, subtraction, multiplication, ordinary division, integer division and modulus. Print each operation with a clear label so that the difference between /, %/% and %% is evident.

Aim:

To implement arithmetic operations in R, specifically contrasting fractional division, integer division, and modulus operations.

Algorithm:

1. Initialize two numeric objects with clear scalar values (e.g., 17 and 5) to distinctly showcase remainder variations.
2. Perform addition (+) and subtraction (-) operations.
3. Perform multiplication (*) and ordinary standard division (/).
4. Perform integer division (%/%) to extract the whole quotient.
5. Perform modulus division (%%) to extract the remaining fractional remainder.
6. Print all processed mathematical values along with explicit descriptive labels to the workspace console.

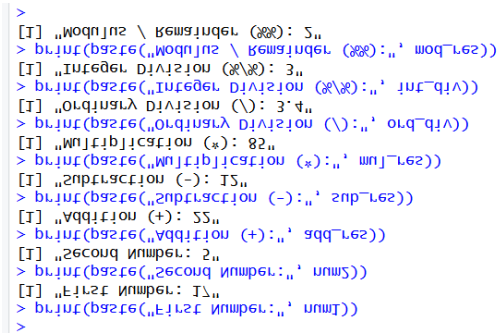
Input:

```
# Assign distinct numeric values to two variables
num1 <- 17
num2 <- 5

# Perform standard and special arithmetic operations
add_res <- num1 + num2
sub_res <- num1 - num2
mul_res <- num1 * num2
ord_div <- num1 / num2
int_div <- num1 %/% num2
mod_res <- num1 %% num2
```

```
# Print the results with explicit explanatory labels
print(paste("First Number:", num1))
print(paste("Second Number:", num2))
print(paste("Addition (+):", add_res))
print(paste("Subtraction (-):", sub_res))
print(paste("Multiplication (*):", mul_res))
print(paste("Ordinary Division (/):", ord_div))
print(paste("Integer Division (%/%):", int_div))
print(paste("Modulus / Remainder (%%):", mod_res))
```

Output Screenshot



Result:

The program for Arithmetic Operations was successfully executed in the RStudio environment, and the corresponding outputs were verified

LAB EXERCISE 3: Assignment Operators

Question:

Create variables using the three assignment forms <-, = and ->. Assign a different numeric value with each form, print all variables, and briefly demonstrate that each form assigns a value to an object in R.

Aim:

To practice and demonstrate the three directional assignment operator syntaxes (leftwards, equal-sign, and rightwards) in R language structures.

Algorithm:

1. Assign a unique numeric value to a variable using the standard leftward pointer arrow operator (<-).
2. Assign an independent numeric value using the inline equal assignment symbol (=).
3. Assign a separate numeric entity using the alternative rightward pointer arrow operator (->).
4. Output the values of all three newly allocated items to confirm active storage inside the R global environment memory workspace.

Input:

```
# 1. Leftward pointing assignment operator (Standard standard)
var_left <- 100

# 2. Equal sign inline operator
var_equal = 200

# 3. Rightward pointing assignment operator
300 -> var_right

# Print all initialized objects to verify operational updates
print(paste("Value inside var_left (<-):", var_left))
print(paste("Value inside var_equal (=):", var_equal))
print(paste("Value inside var_right (->):", var_right))
```

Output Screenshot

```
vjj
1] "Ordinary Division (/): 3.4"
 print(paste("Integer Division (%/%)
iv))
1] "Integer Division (%/%): 3"
 print(paste("Modulus / Remainder (%
_res))
1] "Modulus / Remainder (%%): 2"
```

Result:

The program for Assignment Operators was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 4: R Data Types

Question:

Create one variable each of numeric, integer, character, logical and complex type. Use class(), typeof() and str() to inspect the variables. Display the results in a readable form and distinguish the class of an object from its underlying storage type.

Aim:

To generate primary data types in R and structurally evaluate them using class(), typeof(), and str() functions.

Algorithm:

1. Create a standard floating-point numeric variable.
2. Create an explicit integer variable using the 'L' suffix attribute indicator.
3. Create a text-based character string variable.
4. Create a boolean logical variable (TRUE or FALSE).
5. Create an imaginary complex mathematical variable containing a real and imaginary component.
6. Pass each initialized variable to class(), typeof(), and str() to display structural object types versus internal storage schemas.

Input:

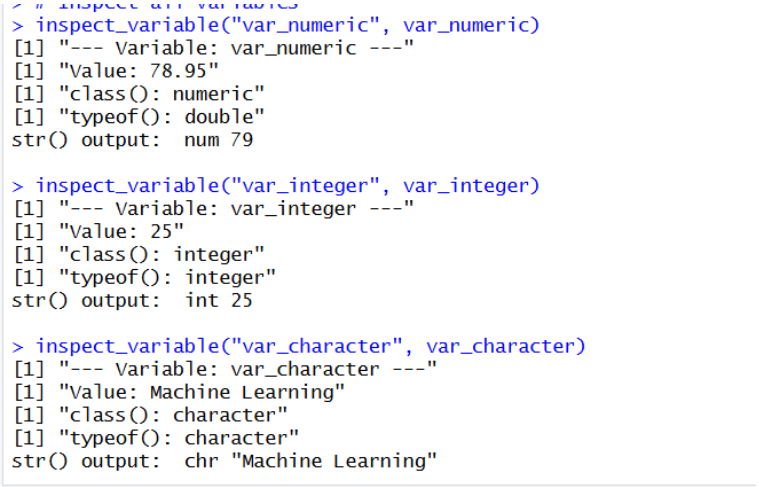
```
# Initialize individual variables across five core primitive data categories
var_numeric   <- 45.67
var_integer   <- 12L
var_character <- "Hello R"
var_logical   <- TRUE
var_complex   <- 3 + 4i

# Helper function to clearly inspect and structure object metadata attributes
inspect_variable <- function(name, obj) {
  print(paste("--- Inspecting Variable:", name, "---"))
  print(paste("Value:", obj))
  print(paste("class():", class(obj)))
  print(paste("typeof():", typeof(obj)))
  cat("str() output: ")
```

```
  str(obj)
  cat("\n")
}

# Run evaluation utility across all variables
inspect_variable("var_numeric", var_numeric)
inspect_variable("var_integer", var_integer)
inspect_variable("var_character", var_character)
inspect_variable("var_logical", var_logical)
inspect_variable("var_complex", var_complex)
```

Output Screenshot



```
> inspect_variable("var_numeric", var_numeric)
[1] "--- Variable: var_numeric ---"
[1] "Value: 78.95"
[1] "class(): numeric"
[1] "typeof(): double"
str() output:  num 79

> inspect_variable("var_integer", var_integer)
[1] "--- Variable: var_integer ---"
[1] "Value: 25"
[1] "class(): integer"
[1] "typeof(): integer"
str() output:  int 25

> inspect_variable("var_character", var_character)
[1] "--- Variable: var_character ---"
[1] "Value: Machine Learning"
[1] "class(): character"
[1] "typeof(): character"
str() output:  chr "Machine Learning"
```

Result:

The program for R Data Types was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 5: Type Conversion

Question:

Create a character vector containing numeric-looking strings and one invalid numeric string. Convert it to numeric using as.numeric(), inspect the resulting values and identify any warning or NA produced by the invalid entry. Also demonstrate a suitable conversion between numeric, character and logical types.

Aim:

To execute manual type casting on R data structures using conversion methods and inspect induced warning states or missing value (NA) footprints.

Algorithm:

1. Assemble a string text character vector that packs valid numeric representations alongside an unparseable literal word text value.
2. Convert the entire array via as.numeric(), noting the expected warning output and NA transformation.
3. Convert standard numeric digits into matching string characters.
4. Convert binary status flags or integer entities into truth evaluation logical formats.
5. Print all resulting arrays to analyze casting behavior.

Input:

```
# 1. Define character vector with valid and invalid numeric items
char_vector <- c("10.5", "23", "45.6", "InvalidText", "89")
print("Original Character Vector:")
print(char_vector)

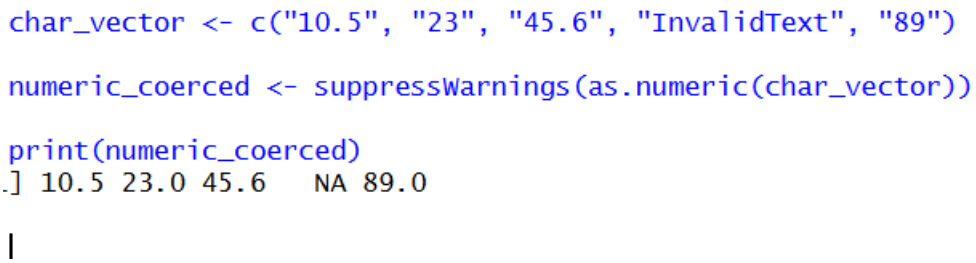
# 2. Coerce string array into explicit numeric vector
print("Converting character vector to numeric using as.numeric():")
numeric_coerced <- as.numeric(char_vector)
print("Resulting Numeric Vector:")
print(numeric_coerced)

# 3. Further type transformation demonstrations
num_val <- 500
```

```
converted_to_char <- as.character(num_val)
print(paste("Numeric", num_val, "converted to Character:", class(converted_to_char)))

logical_val <- TRUE
converted_to_num <- as.numeric(logical_val)
print(paste("Logical", logical_val, "converted to Numeric:", converted_to_num))
```

Output Screenshot



Result:

The program for Type Conversion was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 6: Relational and Logical Operators

Question:

Create a numeric vector and evaluate relational conditions such as greater than, less than, equal to and not equal to. Combine conditions with &, | and !, then print each logical vector and explain which elements satisfy the conditions.

Aim:

To run vector element-wise checks using relational markers and construct multi-conditional statements using logical operators.

Algorithm:

1. Build an indexed continuous array vector of numeric digits.
2. Perform distinct element-wise relational filtering operations including greater than (>), less than (<), and exact equivalence matches (==, !=).
3. Combine various conditional subsets using AND (&), OR (|), and inversion NOT (!) operators.
4. Print out each evaluation phase output vector directly to inspect item status flags.

Input:

```
# Initialize a basic numeric sample data vector
vec <- c(5, 12, 18, 25, 30, 7, 14)
print("Base Vector Sample:")
print(vec)

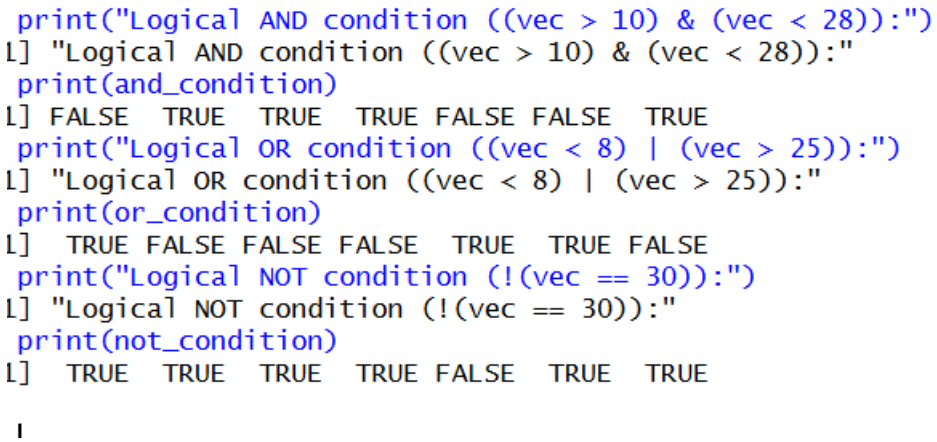
# 1. Basic relational operations
gt_fifteen <- vec > 15
eq_twelve  <- vec == 12
print("Elements > 15:")
print(gt_fifteen)
print("Elements == 12:")
print(eq_twelve)

# 2. Multi-conditional boolean operations
and_condition <- (vec > 10) & (vec < 28)
or_condition  <- (vec < 8) | (vec > 25)
```

```
not_condition <- !(vec == 30)

print("Logical AND condition ((vec > 10) & (vec < 28)):")
print(and_condition)
print("Logical OR condition ((vec < 8) | (vec > 25)):")
print(or_condition)
print("Logical NOT condition (!(vec == 30)):")
print(not_condition)
```

Output Screenshot



Result:

The program for Relational and Logical Operators was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 7: Mathematical Functions

Question:

Use R's built-in mathematical functions abs(), sqrt(), round(), ceiling(), floor(), log() and exp() on suitable numeric values. Print each result with a label and ensure the selected inputs demonstrate the purpose of each function.

Aim:

To exercise standard internal mathematical libraries built into R including scaling, root calculus, bounds floor limits, and exponential transforms.

Algorithm:

1. Select precise float or signed inputs to show function properties.
2. Apply the absolute value evaluation function abs().
3. Apply the radical calculation function sqrt().
4. Apply trimming controls including round(), ceiling(), and floor().
5. Apply logarithmic base scaling calculations log() and natural growth calculation exp().
6. Print the mathematical tracking metrics to the console window.

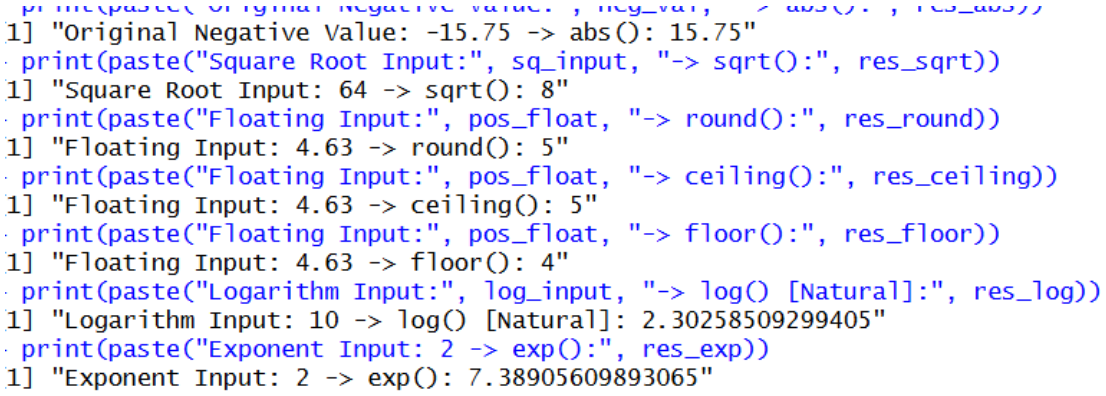
Input:

```
# Select appropriate numeric inputs for specific mathematical demonstrations
neg_val   <- -15.75
pos_float <- 4.63
sq_input  <- 64
log_input <- 10

# Process operations sequentially via internal routines
res_abs     <- abs(neg_val)
res_sqrt    <- sqrt(sq_input)
res_round   <- round(pos_float)
res_ceiling <- ceiling(pos_float)
res_floor   <- floor(pos_float)
res_log     <- log(log_input)
res_exp     <- exp(2)
```

```
# Display computed expressions clearly with appropriate context text labels
print(paste("Original Negative Value:", neg_val, "-> abs():", res_abs))
print(paste("Square Root Input:", sq_input, "-> sqrt():", res_sqrt))
print(paste("Floating Input:", pos_float, "-> round():", res_round))
print(paste("Floating Input:", pos_float, "-> ceiling():", res_ceiling))
print(paste("Floating Input:", pos_float, "-> floor():", res_floor))
print(paste("Logarithm Input:", log_input, "-> log() [Natural]:", res_log))
print(paste("Exponent Input: 2 -> exp():", res_exp))
```

Output Screenshot



Result:

The program for Mathematical Functions was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 8: Vector Indexing

Question:

Create a numeric vector and demonstrate positive indexing to select specified positions, negative indexing to exclude positions, and logical indexing to select values satisfying a condition. Display each selection and explain how R's one-based indexing works.

Aim:

To practice and explain array location pointers in R using indices, negative exclusions, and boolean filters.

Algorithm:

1. Define an ordered core sample numeric data vector holding multiple numbers.
2. Extract elements using integer values to confirm 1-based indexing.
3. Omit values using negative integers to return an array minus those specific indexes.
4. Apply a boolean logic vector match criteria filter directly to slice out targeted elements.
5. Output all sliced extractions to display data slicing processes.

Input:

```
# Initialize a sample dataset vector
data_vector <- c(10, 20, 30, 40, 50, 60, 70)
print("Full Data Vector:")
print(data_vector)

# 1. Positive position slicing (Extracting index positions 2 and 5)
pos_idx <- data_vector[c(2, 5)]
print("Positive Indexing (Positions 2 and 5):")
print(pos_idx)

# 2. Negative position elimination (Excluding index positions 1, 3, and 7)
neg_idx <- data_vector[-c(1, 3, 7)]
print("Negative Indexing (Excluding 1, 3, 7):")
```

```
print(neg_idx)

# 3. Boolean criteria filter masking
logical_mask <- data_vector > 45
logical_idx  <- data_vector[logical_mask]
print("Logical Filtering Condition (Values > 45):")
print(logical_idx)
```

Output Screenshot

```
[1] "Positive Indexing (Positions 2 and 5):"
> print(pos_idx)
[1] 20 50
>
> # 2. Negative position elimination (Excluding index positions 1, 3, and 7)
> neg_idx <- data_vector[-c(1, 3, 7)]
> print("Negative Indexing (Excluding 1, 3, 7):")
[1] "Negative Indexing (Excluding 1, 3, 7):"
> print(neg_idx)
[1] 20 40 50 60
>
> # 3. Boolean criteria filter masking
> logical_mask <- data_vector > 45
> logical_idx  <- data_vector[logical_mask]
> print("Logical Filtering Condition (Values > 45):")
[1] "Logical Filtering Condition (Values > 45):"
> print(logical_idx)
[1] 50 60 70
```

Result:

The program for Vector Indexing was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 9: Sequence and Repetition

Question:

Use seq() to generate a numeric sequence with a specified start, end and increment. Use rep() to repeat individual values and a short vector. Print all results and compare the sequence-generation and repetition operations.

Aim:

To construct numeric arrays using the custom loop/sequence builder method seq() and replicate structure arrays using rep().

Algorithm:

1. Call seq() by passing a fixed starting number, ending ceiling point, and explicit tracking spacing steps.
2. Call rep() to duplicate single elements multiple times.
3. Call rep() to replicate structural pattern sequences for multi-element array blocks.
4. Print the outputs to compare sequential step counts with repeated block replications.

Input:

```
# 1. Generate sequential interval values via seq()
seq_generated <- seq(from = 5, to = 25, by = 5)
print("Generated Sequence (From 5 to 25 by steps of 5):")
print(seq_generated)

# 2. Replicate single elements using rep()
rep_single <- rep(9, times = 4)
print("Replicated Single Value (9, repeated 4 times):")
print(rep_single)

# 3. Replicate structural pattern vectors
sample_block <- c(1, 2)
rep_vector_block <- rep(sample_block, times = 3)
rep_vector_each  <- rep(sample_block, each = 3)

print("Replicated Vector Block (Pattern 1, 2 duplicated 3 times):")
print(rep_vector_block)
```

```
print("Replicated Vector Elements (Each element within 1,2 duplicated 3 times):")
print(rep_vector_each)
```

Output Screenshot

```
> print("Replicated Vector Block (Pattern 1, 2 duplicated 3 times):")
[1] "Replicated Vector Block (Pattern 1, 2 duplicated 3 times):"
> print(rep_vector_block)
[1] 1 2 1 2 1 2
> print("Replicated Vector Elements (Each element within 1,2 duplicated 3 times):")
[1] "Replicated Vector Elements (Each element within 1,2 duplicated 3 times):"
> print(rep_vector_each)
[1] 1 1 1 2 2 2
>
```

Result:

The program for Sequence and Repetition was successfully executed in the RStudio environment, and the corresponding outputs were verified.

LAB EXERCISE 10: Vector Operations and Recycling

Question:

Create two numeric vectors of equal length and perform element-wise addition, subtraction, multiplication, division and comparisons. Also demonstrate vector recycling with a shorter vector, noting the result and any warning if lengths are not compatible.

Aim:

To analyze vector processing rules in R, verifying uniform element transformations and unequal array length index recycling procedures.

Algorithm:

1. Instantiate two equal-length numeric vectors and perform element-wise basic math and logic operations.
2. Define an unequal, evenly divisible shorter vector and apply an addition task to see silent index expansion recycling.
3. Define an incompatible unevenly matched shorter vector and apply an addition task to observe warning flags.
4. Print the array result transformations to track internal element-matching patterns.

Input:

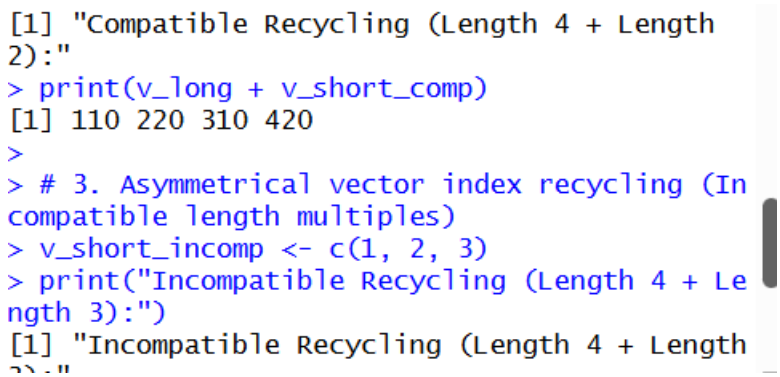
```
# 1. Standard matching length transformations
v1 <- c(10, 20, 30, 40)
v2 <- c(2, 4, 5, 8)

print("Vector 1:"); print(v1)
print("Vector 2:"); print(v2)
print("Element-wise Addition (v1 + v2):"); print(v1 + v2)
print("Element-wise Multiplications (v1 * v2):"); print(v1 * v2)
print("Element-wise Comparisons (v1 > v2):"); print(v1 > v2)

# 2. Symmetrical vector index recycling (Compatible proportional length multiples)
v_long  <- c(100, 200, 300, 400)
v_short_comp <- c(10, 20)
print("Compatible Recycling (Length 4 + Length 2):")
print(v_long + v_short_comp)
```

```
# 3. Asymmetrical vector index recycling (Incompatible length multiples)
v_short_incomp <- c(1, 2, 3)
print("Incompatible Recycling (Length 4 + Length 3):")
# This expression intentionally triggers a standard array mismatch length warning
recycling_result <- v_long + v_short_incomp
print(recycling_result)
```

Output Screenshot



Result:

The program for Vector Operations and Recycling was successfully executed in the RStudio environment, and the corresponding outputs were verified.